DTU Compute
Department of Applied Mathematics and Computer Science

Machine Learning for Semi-Automated Annotations of the Linear A Inscriptions

Bachelor Thesis

DTU Compute
Department of Applied Mathematics and Computer Science
Technical University of Denmark

Richard Petersens Plads
Building 324
2800 Kongens Lyngby, Denmark
www.compute.dtu.dk

Preface

This Bachelor's thesis was prepared at the Department of Applied Mathematics and Computer Science at the Technical University of Denmark in fulfillment of the requirements for acquiring a Bachelor of Science in Engineering (BSc Eng) degree in General Engineering (Cyber Systems).

Kongens Lyngby, March 16, 2026

Frederik W. L. Christoffersen

Abstract

This thesis is in the workings... [Write something when finished with all thesis chapters]

Acknowledgements

Frederik W. L. Christoffersen
General Engineering (Cyber Systems), DTU
Author of this thesis

Morten Mørup
Professor, DTU Compute
Main supervisor

Ester Salgarella
Research Fellow, AIAS Aarhus / University of Cambridge
Co-supervisor, external collaborator, and creator of SigLA

Lukas Esterle
Associate Professor, Aarhus University
Co-supervisor

Generative AI tools were occasionally used as support for brainstorming, linguistic editing, code exploration, and research during the development of this project.

Contents

Acronyms

SAM Segment Anything Model

Cefael Collections de l'École française d'Athènes en ligne

GORILA Godart and Olivier, Recueil des inscriptions en linéaire A

PDR Project Definition Report

IPR Intellectual Property Rights

GMM Gaussian Mixture Model

IoU Intersection over Union

Dice Dice-Sørensen coefficient

TP True Positive

TN True Negative

FP False Positive

FN False Negative

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Background

When the British archaeologist Arthur Evans in the beginning of the 20th century went to excavate Knossos on Crete, he found a huge amount of clay tablets written in a previously unknown linear script. Some documents were clearly of a newer script, which he called *Linear B*, while others were of an older one, which he named *Linear A*. Some fifty years later, the British amateur archaeologist Michael Ventris finally succeeded in deciphering the newer script Linear B, which he proved to be representing Mycenaean Greek, the earliest attested form of Greek. But the same hypothesis did not hold for the older script Linear A, whose encoded language remained unknown, still yet to be deciphered.

A key obstacle to its decipherment has been the small size of its corpus, lately consisting of only around 2,500 inscriptions (about 1,400 if only including those well-preserved enough to be readable) - significantly smaller than that of its younger counterpart that now spans more than 4,500 inscriptions (which are comparatively longer and better preserved).[1,2] But new inscriptions are still being discovered, with the corpus most recently extended in 2025 by over 100 new documents.[3,4]

Another challenge has been the lack of digital representations of these inscriptions, in a form suitable for statistical analysis. While photographs with accompanying drawings of nearly the entire corpus are available online through Collections de l'École française d'Athènes en ligne (Cefael) (see **EtudesCretoises21**), these are only available in print form. This

[1] **Salgarella2020**.
[2] **Salgarella2025**.
[3] **RILA-S1**.
[4] **CNRLinearACorpus2025**.

motivated the development of the palæographical[5] database *The Signs of Linear A* (SigLA), which seeks to ”[develop] a systematic, exhaustive and user-friendly open access database of all Linear A inscriptions [...] in order to carry out statistical and palæographic analyses within the epigraphic corpus” (**SalgarellaCastellanSigLA2020**). However, the last addition to SigLA was in 2021, and the project has so far managed to document just 772 of all currently known inscriptions.[6,7]

1.1.1 A problem of manual digitization

The drawings currently stored on SigLA were made by Ester Salgarella using the painting tool Krita. For each document, she manually created her own annotated drawings based on the originals from Cefael. From these, she proceeded to create layered Krita files with metadata for each sign, which could then at last be imported to SigLA.[8] According to her (personal communication), this has been a highly time-consuming undertaking. And the necessity of this kind of labor-intensive work has arguably been the biggest reason to why SigLA still does not contain the entire available corpus.

The aim of this project is therefore to investigate whether parts of this process can be automated. More specifically, whether some kind of semi-automatic segmentation tool can accelerate the workflow, while maintaining high-quality outputs that are still compatible with SigLA’s database.

1.1.2 A problem of small data

The small size of the Linear A corpus and the irregularities in quality of its documents is a huge problem for the application of deep learning approaches, which typically require large amounts of data to perform well. Consequently, this project will instead seek to make use of generalized

[5]The term ”palæographical” refers to the study of ancient writing systems.
[6]**SigLAAboutPage**.
[7]**SigLASearchDocuments**.
[8]**SalgarellaCastellanSigLA2020**.

machine learning and computer vision, in particular Meta's state-of-the-art interactive segmentation tool, the Segment Anything Model (SAM) (**sam**), which is pre-trained on a massive and diverse dataset of over 1 billion masks. SAM's generalization and interactivity (allowing user-placed markers that guide the model) might thus allow for semi-automatic annotation of Linear A inscriptions without the need for large amounts of training data, which is unfortunately not available in this context.[9]

[9] By the "interactivity" of SAM, what is meant is NOT any built-in user interface in the form of a GUI, but rather the possibility of prompt-based placement of rectangles of focus, and points of additions and deletions that guide the model's prediction.

1.2 Prior work

This thesis places itself at the intersection of computer vision, digital humanities, and epigraphy (the study of inscriptions). Since it is a thesis of engineering, computer vision will remain the primary focus, both in terms of the research questions, and the methods used to answer them. As a result, the other two fields will merely provide the context and motivation for the research. The prior work and initial literature review in focus will thus concentrate on: 1. the relevant theory and state-of-the-art within computer vision (SAM, etc.), 2. the palæographical and digital humanities work that this project will seek to support (SigLA), and 3. the overall context thereof (Linear A).

1.2.1 Research context

For the context of Linear A and its corpus, the background section above can be referred to, citing the sources for: its size (**Salgarella2020**) and (**Salgarella2025**), its new addition (**RILA-S1**), and where it's accessed (**EtudesCretoises21**). And for the epigraphical and digital humanities work, the background section can also be referred to, citing the sources for SigLA: its paper (**SalgarellaCastellanSigLA2020**), its about page (**SigLAAboutPage**), and its document repository (**SigLASearchDocuments**). Here, additional context can also be provided by Ester Salgarella herself, as she is a co-supervisor of this project, and the main person behind SigLA.

1.2.2 Computer vision literature

As for the relevant theory and state-of-the-art within computer vision, the literature there can be further split into three parts; 1. sources of general image processing, 2. sources concerning SAM and its implementation, and 3. related computer vision script segmentation work for comparison.

For image processing, a general theory book (**DigitalImageProcessing**) can be cited, as well as more method-specific papers (when other methods are used) like Otsu thresholding (**Otsu1979**), and of course the OpenCV

Python documentation (**opencv_imgproc_docs**) for short implementation reference. Then, for SAM, these are: its paper (**sam**), its lighter implementation MobileSAM (**mobile_sam**) - and the newest version of that one MobileSAMv2 (**mobile_sam_v2**). Lastly, for related segmentation work there exists a paper using deep learning (in contrast to the methodology of this thesis) obtaining results better than state-of-the-art (**DeepLearningSegmentation**), and a lighter EU-funded historical document segmentation tool *dhSegment* (**dhsegment**) also using deep learning.

1.3 Problem statement

As previously stated, the aim of this project is to investigate whether parts of the process of creating annotated drawings for SigLA can be automated. More specifically, whether some kind of semi-automatic segmentation tool using interactive and generalized computer vision - specifically the Segment Anything Model (SAM) - can accelerate the workflow, while maintaining high-quality outputs that are still compatible with SigLA's database. But what would be the relative segmentation quality of such a tool? How automatic would it be exactly? And by how much could it improve the efficiency of the workflow? For a decomposition of the problem statement into its three research questions - each accompanied by their respective operationalization - see the following subsection:

1.3.1 Research questions

1. **Segmentation performance**: How well does a SAM-based segmentation approach perform quantitatively in terms of segmentation accuracy, compared to simple classical image processing baselines when applied to Linear A document images? [Operationalization: SAM-based masks will be compared to classical image-processing baselines using overlap metrics (e.g. IoU/Dice) against the ground truth segmentations derived from SigLA's Krita files for a minimum of 25 documents.]

2. **Degree of automation**: To which extent can user interaction be reduced by a SAM-based semi-automatic segmentation workflow, while still maintaining high-quality outputs? [Operationalization: The number of user interactions (e.g. clicks) required to achieve a certain level of segmentation quality (e.g. IoU/Dice) will be measured against ground truth (SigLA's Krita files) for a minimum of 25 documents.]

3. **Workflow efficiency**: How much reduction in annotation time can be achieved using the proposed tool compared to fully manual digitization, under controlled experimental conditions? [Operationalization: The time it takes Ester Salgarella to annotate manually compared to being assisted by the proposed segmentation tool will be measured for a minimum of 5 documents.]

1.4 Empirical considerations

For the evaluation of the solution, a dataset will be compiled together with Ester Salgarella of at least 25 images of original documents from the corpus, selected and cropped from Cefael. These images are subject to Intellectual Property Rights (IPR) issues, and can thus not be displayed publicly if not through Cefael. However, they can still be used for the development and evaluation of the tool (and can be bought if necessary for show-casing within the thesis). For direct reference to the primary source of data, the specific links to the Cefael online webpages for each document used might have to be cited.

As ground truth, their corresponding digitized layered drawings, stored as Krita (.kra) files on SigLA (provided by Ester Salgarella) will be used and aligned for comparison with tool outputs. The ground truth data might then have to be cleared of any marked "deleted signs" or other annotation elements that our model is not intended to reproduce.

The solution will then be built as a pipeline, consisting of pre-processing, semi-automatic segmentation, post-processing, and export (e.g. as Krita files). At last, the primary deliverable of this project will be delivered to the external partner and co-supervisor Ester Salgarella in the form of a proof-of-concept semi-automatic annotation tool. All supporting components, including evaluation results, and analysis will be documented and presented in the written thesis.

1.5 Impact – innovation and application

The work of this thesis will present itself as a first proof-of-concept for the use of modern computer vision to accelerate the expansion of the SigLA database, supporting the further digitization of the Linear A corpus. Its main contribution will be a semi-automatic annotation tool, which might prove useful to epigraphers for generating segmentations of original document images, as they can then be completed and refined more easily. This approach could potentially also be applied to Linear B, as it is the whole field of Aegean studies that seem to suffer from severe under-digitization of the primary evidence, which is a huge problem for the scientific investigations of scholars. Hopefully, it will inspire further research and development within the field of digital epigraphy, particularly in the application of generalized and interactive computer vision approaches such as SAM on Linear A, and perhaps even in the broader context of use on small corpus ancient writing systems where the lack of big data is ill fit for approaches using deep learning.

[TODO: FIX THE INTRO AND FIT IT TO THE THESIS]

1.6 Overview of the thesis

The rest of this thesis is structured into five main chapters (Methodology, Data, Results, Discussion, Conclusion), all split up into sections concerning each of the three research questions.

CHAPTER 2
Methodology

2.1 Common methodology

[TODO: WRITE A C]

2.1.1 Image processing theory

[SHOULD THIS SECTION EXIST? IF SO, I SHOULD FINISH IT]

The basics of spatial filtering can be described by the following equation:

$$g(x, y) = T[f(x, y)]$$

, where $f(x, y)$ (also denoted as $I(x, y)$) is the input image, $g(x, y)$ is the output image.[1]

2.1.1.1 Binary image thresholding

$$B(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T \\ 0 & \text{otherwise} \end{cases}$$

[2]

[1] **DigitalImageProcessing**.
[2] **DigitalImageProcessing**.

2.1.1.2 Convolutional kernels

[3]

$$(w \star I)(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(i,j) I(x-i, y-j)$$

where w is the convolution kernel (window) of size $m \times n$, and $a = (m-1)/2$ and $b = (n-1)/2$ are the half-width and half-height (assuming odd dimensions), respectively.[4]

2.1.2 Development environment and tools

[TODO: EXTEND THIS SECTION] Python 3.10 was used for all code development (because it has such a big library of image processing functions and machine learning model implementations), with the following libraries:

- OpenCV for image processing and computer vision tasks.
- NumPy for numerical operations and array manipulations.
- Matplotlib for data visualization.
- *Add more!*

Krita was used for manual image registration, making sure the raw image ground truth pairs were properly aligned.

[3] **DigitalImageProcessing**.
[4] **DigitalImageProcessing**.

2.1.3 Data assumptions (brief reference to Data chapter)

2.1.4 General evaluation metrics

2.1.5 Experimental conventions

2.2 Segmentation quality

Here, the one-shot segmentation quality of SAM will be evaluated against a set of baseline models, using a variety of performance metrics.
[Add some context of RQ1 here, and how the question will be operationalized in bigger picture (though more detail than in introduction).]

2.2.1 Baseline model configuration

The baseline models against which the tool will be evaluated (from simplest to most advanced) are:

- Global thresholding (Otsu's method)
- Local adaptive thresholding (Gaussian kernel)
- Edge-based segmentation (Canny edge detection with region filling)
- Energy-based segmentation (GrabCut)

Together, they cover a wide range of classic segmentation techniques that might be used for binary foreground extraction. What follows is a brief description of each of these methods, including their characteristics, how they work, and their implementation details. [TODO: ADD MORE IMPLEMENTATION DETAILS BESIDES PURE THEORY] [SHOULD I REDUCE THEORY? HOW LONG SHOULD THESE BE? PROBABLY NOT MORE THAN A PAGE FOR EACH BASELINE]

2.2.1.1 Global thresholding (Otsu's method)

This is a global thresholding method from gray-level histograms originally proposed by **Otsu1979**. As the simplest of the baselines, it represents the minimum acceptable segmentation quality, since it only takes the global pixel intensity into account, with no assumptions about any spatial relationships between pixels. It might therefore perform poorly on images with erosion, uneven lighting, and differences in surface texture[5], which are the exact characteristics of the Linear A inscriptions to be segmented.

Here is how it works: First, let pixels of the raw image be divided into L shades of gray $[1, 2, ..., L]$. Then, let each pixel be assigned one of two classes C_0 or C_1, guided by a threshold T, where C_0 contains the shades $[1, 2, ..., T]$, and C_1 contains the rest $[T + 1, T + 2, ..., L]$. Otsu's method then works by performing an exhaustive search over all L gray levels to find an optimal threshold T^* that results in a maximum between-class variance σ_B^2:

$$T^* = \underset{1 \leq T < L}{\arg\max}\ \sigma_B^2(T)$$

The between-class variance σ_B^2 can then be defined by a function of the respective total probabilities of the two classes (ω_0 and ω_1) and the mean gray level value per class (μ_0 and μ_1):

$$\sigma_B^2(T) = \omega_0(T)\omega_1(T)[\mu_1(T) - \mu_0(T)]^2$$

For the implementation of Otsu's method in OpenCV, the following parameters are used:

- `cv2.THRESH_OTSU` for the thresholding type, and
- `cv2.THRESH_BINARY` for the binary thresholding method.

2.2.1.2 Local adaptive thresholding (Gaussian kernel)

Using local adaptive thresholding is an advantage in cases where the image has varying light conditions (which is the case for the Linear A inscriptions), as the filter adapts to local lighting. This is due to the fact

[5] **DigitalImageProcessing**.

that the computed threshold T is not global, but instead computed for each pixel based on its local neighborhood.

Mathematically, T is a function of a given pixel position (x, y) usually computed by some kind of weighted sum of the neighborhood. The generated binary mask B is then defined as:[6]

$$B(x,y) = \begin{cases} 1 & \text{if } I(x,y) > T(x,y) \\ 0 & \text{otherwise} \end{cases}$$

To compute $T(x, y)$, there are a few weighting functions to choose from. But for this baseline, the Gaussian kernel is used, as it tends to be better at handling noise, since it assigns weights based on distance from the kernel center. The pixels closer to the center will have a higher weight, while those farther away will have a lower weight, which limits the influence of distant noisy pixels.

With a Gaussian kernel G_σ, the threshold $T(x, y)$ is computed as a Gaussian-weighted sum of local intensities subtracted by a constant C. Using convolution ($\star$), this can be expressed as:[7]

$$T(x,y) = (G_\sigma \star I)(x,y) - C$$

The weight at a given pixel offset (i, j) from the kernel center (where α is a normalization constant ensuring that the sum of the kernel elements equals 1, and σ is the standard deviation of the weights) is then defined as follows:[8]

$$G_\sigma(i,j) = \alpha \cdot \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right)$$

The OpenCV implementation[9,10] uses the following parameters:

[6] **DigitalImageProcessing**.

[7] **opencv_adaptive_threshold**.

[8] **DigitalImageProcessing**.

[9] **opencv_gaussian_kernel**.

[10] The Gaussian kernel is *separable* and *isotropic*, meaning that it can be implemented (like done in OpenCV) as two consecutive 1-D convolutions, reducing computational complexity from $O(n^2)$ to $O(n)$ for a kernel of size $n \times n$. (**DigitalImageProcessing**).

- `cv2.ADAPTIVE_THRESH_GAUSSIAN_C` for the thresholding type, and
- `cv2.THRESH_BINARY` for the binary thresholding method.

2.2.1.3 Edge-based segmentation (Canny edge detection with region filling)

This is a method that by itself only segments the edges of an image, excluding the enclosed regions within those edges. Here is how it works: First, the input image is smoothed using a Gaussian filter to reduce noise (using the Gaussian kernel as defined in the previous section):

$$I_s(x, y) = (G_\sigma \star I)(x, y)$$

Then, for the image gradient of each pixel, the magnitude $M_s(x, y)$ and the orientation $\theta_s(x, y)$ are computed:[11]

$$M_s(x, y) = \| \nabla I_s(x, y)\| = \sqrt{\left(\frac{\partial I_s}{\partial x}\right)^2 + \left(\frac{\partial I_s}{\partial y}\right)^2}$$

$$\theta_s(x, y) = \arctan\left(\frac{\partial I_s/\partial y}{\partial I_s/\partial x}\right)$$

Next, *non-maxima suppression*[12] is applied to retain only local maxima of $M_s(x, y)$ along the gradient direction $\theta_s(x, y)$, resulting in thinner edges. This is at last followed by *hysteresis thresholding*[13], classifying pixels as strong, weak, or non-edges with a low threshold T_l and a high threshold T_h, only preserving weak edges if connected to strong edges.
[Add details about region filling here.]
The OpenCV implementation[14,15] uses the following parameters:

- `cv2.Canny` for the edge detection method.

[11] **DigitalImageProcessing**.
[12] **DigitalImageProcessing**.
[13] **DigitalImageProcessing**.
[14] **opencv_gaussian_kernel**.
[15] The Gaussian kernel is *separable* and *isotropic*, meaning that it can be implemented (like done in OpenCV) as two consecutive 1-D convolutions, reducing computational complexity from $O(n^2)$ to $O(n)$ for a kernel of size $n \times n$. (**DigitalImageProcessing**).

2.2.1.4 Energy-based segmentation (GrabCut)

GrabCut - the most advanced of the baselines considered - is included, as it is an interactive segmentation method, which makes for a good comparison to SAM. It is also good at producing spatially coherent masks with little large-grained noise, which is useful on Linear A inscriptions, where erosion usually leads to noisy masks even after noise reduction. Here is how it works: For energy-based segmentation methods, optimal segmentation is achieved by minimizing a cost (energy) function E that combines a data term D and a smoothness term S, given an input image I, a binary segmentation mask B, and a set of parameters θ:[16]

$$E(B, I, \theta) = D(B, I, \theta) + S(B, I)$$

In the case of GrabCut (**Rother2004GrabCut**), the algorithm is interactively initialized by the user, who provides a bounding box around the object of interest. Pixels outside this box are marked as definite background, while pixels inside it are marked as initially unknown. To further guide the segmentation, the user also has the option to provide an additional number of pixels as definite foreground ($B_n = 1$), and definite background ($B_n = 0$).

The set of parameters θ then define two *Gaussian Mixture Models (GMMs)* (probabilistic models each consisting of a weighted sum of several Gaussian distributions) that over iterations will be tuned to model the foreground and background pixel distributions. These are used to compute the data term D whose energy contribution decreases the better B matches the GMMs of foreground and background. The smoothness term S then penalizes neighboring pixels that have different labels, encouraging spatial coherence in the segmentation.

At last, the optimal segmentation B^* is found iteratively, by alternating between optimizing the GMM parameters θ to model B, and minimizing the energy function with respect to B:

$$B^* = \arg\min_B E(B, I, \theta)$$

[THIS CLASSICAL INTERACTIVE SEGMENTATION METHOD MIGHT FOLLOW SAM INTO RQ2]

[16] This is a simplified version of the energy function that is used by GrabCut.

[MAYBE IT SHOULD BE GIVEN MORE IMPORTANCE?]

2.2.2 State-of-the-art model configuration

2.2.2.1 Model architecture

Raw image → Automatic box prompts → MobileSAMv2 → Binary mask

2.2.2.2 Seed extraction (Autoboxes)

Even though the MobileSAMv2 implementation already makes use of an automatic box prompt, it does so from a YOLO model trained on the same dataset as SAM. Those box prompts are generalized, and work best on natural images, not on image inscriptions. Therefore, there is to be made two versions of MobileSAMv2 for comparison: one with the native box prompts *mSAM*, and one with box recognition done by contours in the image *mSAMautobox*. [Please improve this draft sub sub section]

2.2.2.3 Segmentation (MobileSAMv2)

MobileSAMv2 (**mobile_sam_v2**)

2.2.3 Evaluation protocol and performance metrics

2.2.3.1 Choice of metrics

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Dice = F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

$$IoU = \frac{Dice}{2 - Dice} = \frac{TP}{TP + FP + FN}$$

Dice is more commonly used in cases where have heavy class imbalances (e.g. lots of background compared to regions segmented), or the objects to be segmented are small and thin. It is naturally more foregiving than IoU, which tends to be rather strict and used to evaluate segmentation where small displacements really matter. Therefore, this thesis will measure segmentation performance and report it using Dice. (**Metrics2025**)

2.2.3.2 Statistical tests

Non-parametric Friedmann test + Bonferroni corrected Wilcoxon tests. As opposed to parametric ANOVA and paired t-test. (**Rainio2024**)

2.3 Interactive semi-automation

Here, the interactivity and quality of human-guided SAM (and GrabCut for comparison?) will be evaluated against the one-shot results from Section 2.2. [Add some context of RQ2 here, and how the question will be operationalized in bigger picture (though more detail than in introduction).]

2.3.1 Interactive tool architecture

2.3.2 Experimental design for automation evaluation

2.3.3 Definition of automation metrics

2.4 Workflow efficiency

[Add some context of RQ3 here, and how the question will be operationalized in bigger picture (though more detail than in introduction).]

2.4.1 Definition of workflow efficiency metrics

2.4.2 Experimental design of the user study

2.4.3 Analysis of results

CHAPTER 3

Data

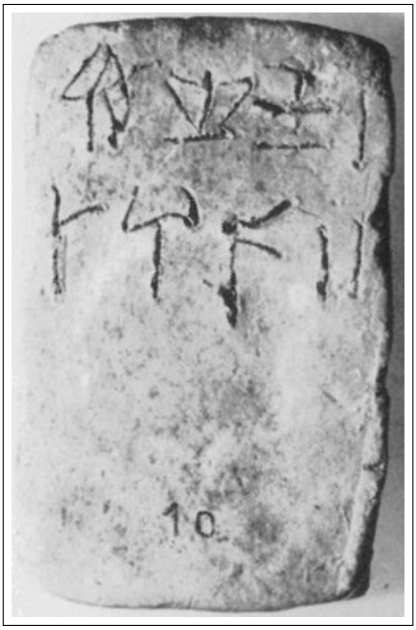
3.1 Sources

The dataset used in this thesis was constructed from photographs of raw Linear A inscriptions, and their corresponding ground truth annotations. Cefael (**EtudesCretoises21**) was chosen as the source for raw document images, since it is the only online and widely publicly available source of the printed five volume corpus - Godart and Olivier, Recueil des inscriptions en linéaire A (GORILA). As all raw document images on the pages of this corpus are accompanied by annotated drawings, an argument could be made to make use of those as ground truth annotations. However, annotations provided by Ester Salgarella from SigLA (**SalgarellaCastellanSigLA2020**) were chosen instead, as they are more up to date, and are already in a binary mask format that is suitable for comparison with segmentation model outputs.

What follows in this chapter is a detailed description of this dataset, documenting its characteristics, its construction process, and its limitations and biases.

3.1.1 Characteristics and construction

The dataset was made to consists of 32 chosen image pairs of raw documents and their corresponding ground truth annotations. Only inscriptions with clay as writing support were included, as they are the most common and representative of the Linear A corpus, and are the only ones for which SigLA ground truth annotations were already available (IS THIS TRUE?). The raw document images from Cefael are screenshots of images within the scanned printed edition of the corpus, and thus are not of the highest quality, but they are the only widely publicly available images of the Linear A inscriptions.



((a)) Raw image (HT7b)

((b)) Ground truth (HT7b)

Figure 3.1: Example of a raw inscription image and its corresponding ground truth annotation.

3.1.2 Difficulty classification

The dataset was categorized into three difficulty levels (easy, medium, hard) based on the visual quality of the raw images and the clarity of the inscriptions. This classification was performed manually by inspecting each image and considering factors such as contrast, noise, and the presence of tablet breakages or general damage to the inscriptions. Concretely, the following criteria were used to classify each difficulty set:

X

- Easy: labeled as such for documents with clear ingravings, good contrast, minimal noise, and constant lighting.
- Hard: labeled to those with poor contrast, significant noise, and substantial damage making it hard even for the naked eye to read the inscriptions.
- Medium: labeled to all those inbetween the qualities of easy and hard.

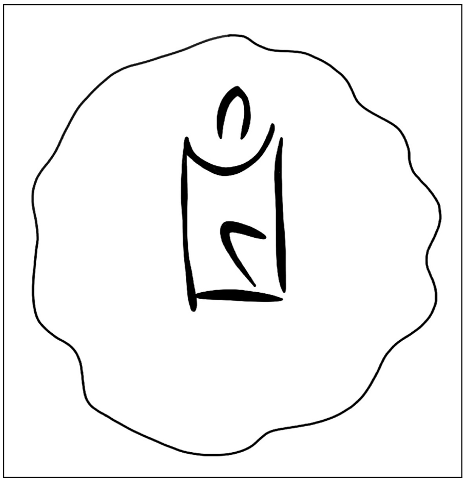
Figure 3.2: Criteria for classifying the dataset into easy, medium, and hard difficulty levels.

3.1.3 Image registration and preprocessing

To obtain any kind of meaningful comparison between model outputs and ground truth, the raw images and their ground truth annotations first had to be aligned and registered to each other. This was done manually in Krita by downscaling the ground truth images to the individual scales of their raw counterparts, and aligning by translation and rotation, using the raw images as background reference.

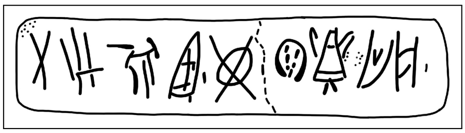
3.2 Limitations and biases

((a)) Easy KHWc2104 (raw)

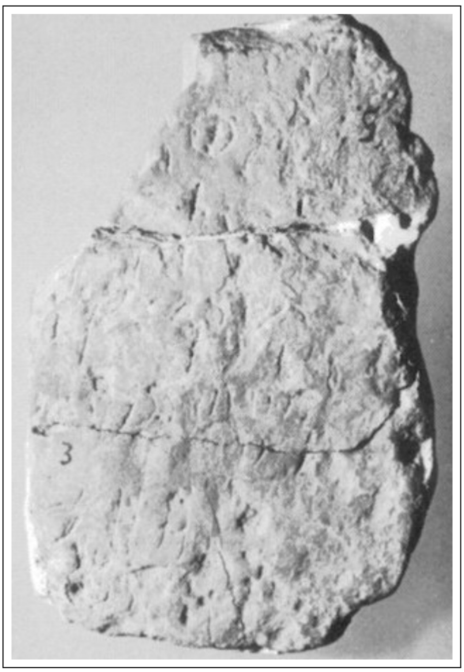


((b)) Easy KHWc2104 (gt)

((c)) Medium MA1a (raw)



((d)) Medium MA1a (gt)



((e)) Hard HT3 (raw)

((f)) Hard HT3 (ground truth)

Figure 3.3: Examples of images with different levels of segmentation difficulty: easy, medium, and hard. For each case, the raw image (left) and the corresponding ground truth annotation (right) are shown.

CHAPTER 4

Results

4.1 Segmentation performance

Table 4.1: Segmentation IoU on easy images.

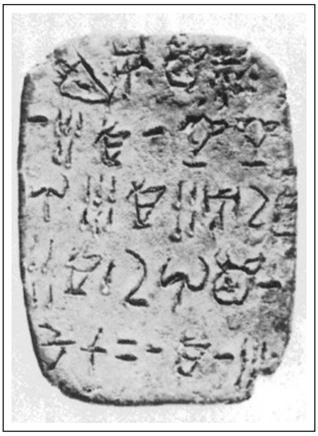
Model	Mean IoU	σ	Standard error	n
Otsu	0.054	0.02	0.006	11
Gaussian	0.115	0.045	0.014	11
CannyFill	0.023	0.027	0.008	11
GCautobrush	0.122	0.06	0.018	11
mSAM	0.013	0.009	0.003	11

$$v = \frac{\left(\frac{s_1^2}{n} + \frac{s_2^2}{n}\right)^2}{\frac{\left(\frac{s_1^2}{n}\right)^2}{n-1} + \frac{\left(\frac{s_1^2}{n}\right)^2}{n-1}}$$

$$t = \frac{(\delta)}{\sqrt{n}}$$

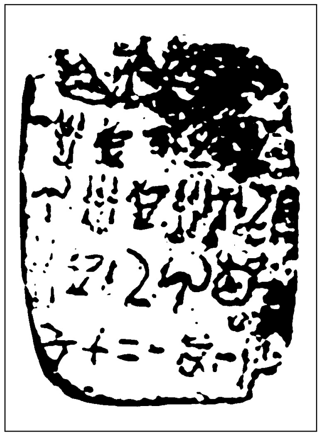
4.2 Automation evaluation

4.3 Workflow efficiency evaluation

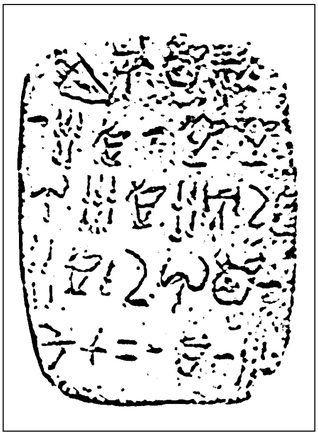


((a)) Raw

((b)) Ground Truth

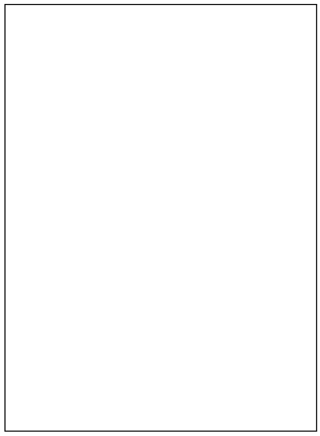


((c)) Otsu

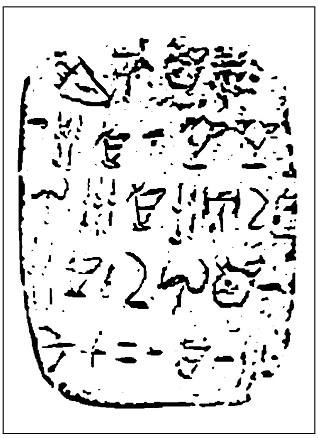


((d)) Gaussian

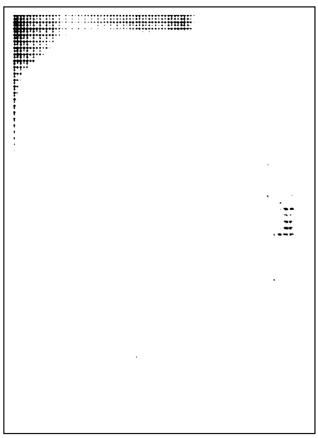
((e)) CannyFill



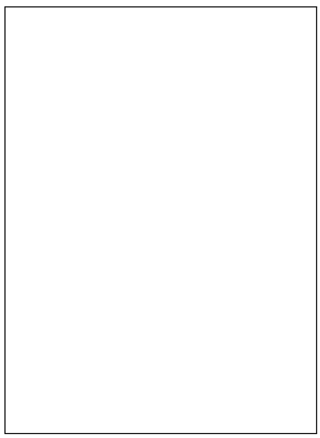
((f)) GCautobox



((g)) GCautobrush



((h)) mSAM



((i)) mSAMautobox

Figure 4.1: Comparison of segmentation outputs from different models against the raw image and ground truth.

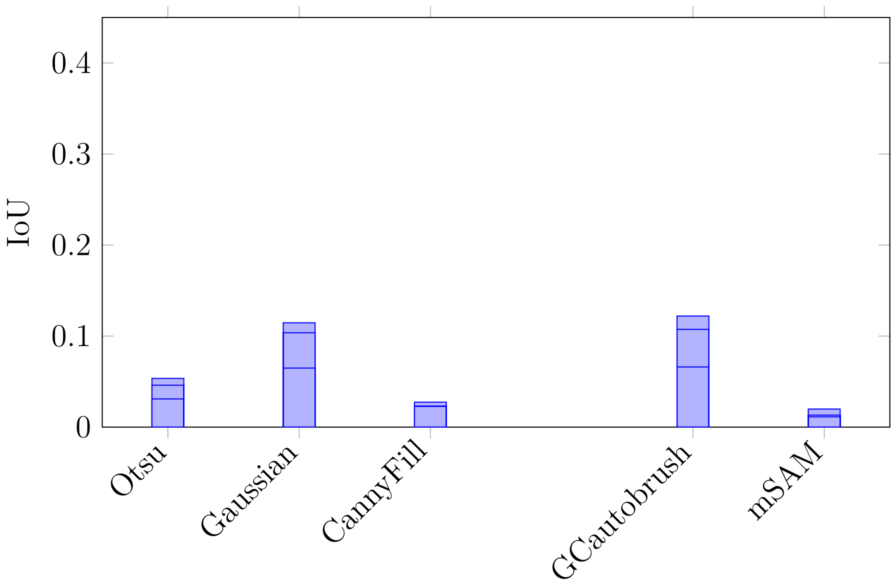


Figure 4.2: IoU performance per model. Each bar shows the overall performance, while the horizontal lines within each bar indicate the IoU values obtained for the easy, medium, and hard difficulty subsets.

Table 4.2: Segmentation IoU on easy images.

Model	Mean IoU	σ	Standard error	n
Otsu	0.054	0.02	0.006	11
Gaussian	0.115	0.045	0.014	11
CannyFill	0.023	0.027	0.008	11
GCautobrush	0.122	0.06	0.018	11
mSAM	0.013	0.009	0.003	11

CHAPTER 5

Discussion

5.1 Segmentation ability

Florence 2 (**Wang2025OBIFloSAM**)

5.2 Automation and interactivity

5.3 Workflow impact

CHAPTER 6

Conclusion

6.1 Summary of findings per research question

6.1.1 RQ1: Segmentation performance

6.1.2 RQ2: Degree of automation

6.1.3 RQ3: Workflow efficiency

6.2 Overall contribution

6.3 Final remarks

CHAPTER 7

Future Work

7.1 Methodological extensions

7.2 Dataset expansion and diversification

7.3 Further automation opportunities

APPENDIX A

An Appendix

A.1 Raw segmentation performance

Table A.1: Segmentation Dice score on easy images.

((a)) Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GCautobrush	Gaussian	Otsu	mSAM
HT118	0.072	0.169	0.162	0.1	0.002
HT12	0.024	0.26	0.259	0.176	0.048
HT154B	0	0.146	0.147	0.07	0.032
HT17	0.021	0.23	0.215	0.072	0.03
HT22	0	0.192	0.187	0.13	0.057
HT26a	0.021	0.263	0.256	0.11	0.001
HT40	0.092	0.159	0.161	0.074	0.029
HT7a	0.108	0.129	0.136	0.109	0.02
HT7b	0.144	0.179	0.162	0.073	0.026
HT90	0	0.179	0.181	0.131	0.012
KHWc2104	0	0.44	0.37	0.067	0.027

Table A.2: Segmentation Dice score on medium images.

((a)) Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GCautobrush	Gaussian	Otsu	mSAM
HT10a	0.021	0.253	0.249	0.119	0.057
HT16	0.032	0.178	0.159	0.08	0.003
HT2	0.01	0.113	0.127	0.082	0.021
HT4	0.003	0.237	0.239	0.097	0.061
HT6b	0.157	0.188	0.175	0.071	0.004
HT85b	0.063	0.118	0.106	0.058	0.035
HT8b	0.07	0.273	0.267	0.092	0.052
HT94b	0.082	0.083	0.085	0.035	0.017
MA1a	0.083	0.232	0.227	0.146	0.09
PH2	0.003	0.242	0.224	0.095	0.046

Table A.3: Segmentation Dice score on hard images.

((a)) Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GCautobrush	Gaussian	Otsu	mSAM
HT1	0.021	0.117	0.103	0.045	0.049
HT27a	0.094	0.153	0.148	0.056	0.047
HT3	0.025	0.08	0.089	0.054	0.046
HT30	0.039	0.096	0.098	0.05	0.006
HT33	0.044	0.054	0.051	0.016	0
HT52a	0	0.1	0.109	0.059	0.006
HT85a	0.092	0.085	0.097	0.055	0.006
HT8a	0.12	0.206	0.186	0.056	0.041
HT9a	0.002	0.218	0.212	0.137	0.02
HT9b	0	0.117	0.115	0.067	0

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk